

Physical Intelligence $\pi_{0.5}$ on Apex Compute’s Unified Engine

Apex Compute

1 $\pi_{0.5}$

$\pi_{0.5}$ is a vision–language–action policy: camera views and an instruction in, a chunk of future control steps out. Three stages run in sequence: a SigLIP vision encoder, a Gemma-2B prefix that produces a K/V cache, and a Gemma-300M action expert that denoises the action chunk against that cache. The complete FPGA deployment executes all three stages on the accelerator and matches the CUDA reference outcome on all ten closed-loop evaluation prompts.

The prefix accounts for three quarters of the arithmetic (Table 1) and sets the main compute workload. The action expert then refines the action chunk through ten Euler steps, $x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t, \text{KV}_{\text{prefix}})$, using the cached prefix context. This maps naturally to a high-throughput prefix stage followed by a compact, deterministic control stage.

Table 1: Per-inference budget at the model’s true shape. The measured U50 deployment retains an aligned 832-row prefix.

Stage	Shape	Params	GFLOP
Vision (SigLIP ViT)	27 L, $d1152$, 3×256 tok	~ 0.41 B	655.9
Prefix (Gemma-2B)	18 L, $d2048$, 576 effective tok	~ 2.0 B	2332.0
Expert (Gemma-300M)	18 L, $d1024$, 10 steps	~ 0.31 B	71.0
Total		~ 2.7 B	3058.9

2 Porting

The Unified Engine is a bit-matching ASIC IP implemented on an AMD Alveo U50 with 8 GB of DRAM and operating at 366 MHz. Projection and MLP up/down weights use 4-bit IF4. For each quantization block, IF4 selects either INT4 or FP4, whichever produces the lower quantization MSE for that block. All other weights are bf16.

3 Quantization

3.1 IF4

IF4 evaluates signed INT4 and FP4 E2M1 for each 64-element block and keeps the lower-error representation. It therefore matches or improves on either INT4 or FP4 alone, while using the same 4-bit storage. The block choice is encoded in the sign of its bf16 scale, so no side table is required. Table 2 shows IF4 matching the best standalone format on the synthetic extremes and exceeding both standalone formats on the $\pi_{0.5}$ checkpoint groups.

Table 2: Weight-domain SNR (dB) for INT4, FP4, and IF4. Synthetic blocks above, $\pi_{0.5}$ checkpoint group means below.

Block source	INT4	FP4	IF4
Uniform (bounded)	23.12	19.36	23.12
Gaussian	19.35	19.52	20.11
Gaussian + one outlier	11.81	16.50	16.50
Action expert	19.50	19.47	20.25
Prefix LM	19.05	19.47	20.00
Vision encoder	18.31	19.23	19.65
Token embedding	15.39	18.53	18.65

3.2 End-to-end numerical agreement

Table 3 measures mixed-precision agreement against the bf16 reference with identical inputs and noise. The final action reaches 28.47 dB at $\text{MSE } 2.22 \times 10^{-4}$. The closed-loop evaluation in Sec. 3.3 then verifies that the numerical differences preserve the same task outcome on every evaluated prompt.

Table 3: Mixed IF4/bf16 vs. bf16 numerical agreement by section (torch reference, sample 0).

Model section	SNR (dB)	Range (dB)
Vision encoder (3 cameras)	25.39	24.3–26.6
Prefix K-cache (18 layers)	16.57	13.6–19.9
Prefix V-cache (18 layers)	11.40	8.1–17.0
Denoise velocity v_t (10 steps)	31.51	22.2–34.5
End-to-end action (10×7)	28.47	$\text{MSE } 2.22 \times 10^{-4}$

3.3 LIBERO

LIBERO is a tabletop manipulation benchmark for language-conditioned policies, scored by closed-loop task success. It provides two camera views; $\pi_{0.5}$ takes three, so the third slot is zero-filled and masked.

An episode is many inferences. The simulator executes the first ten steps of each chunk and requests the next, so one episode is a dozen or more full passes on hardware. Table 4 compares the FPGA policy with the CUDA reference over ten prompts with the same task definitions, initial seed, and fixed denoise noise. This paired test measures deployment parity under matched initial conditions; it is not intended as a benchmark-wide estimate of task success.

Table 4: LIBERO closed-loop outcomes, FPGA (IF4/bf16) vs. CUDA (bf16): 10 prompts, one episode each, fixed seed and noise. ✓ success, × failure, number = episode length in sim steps. Δ = FPGA – CUDA steps on successes; failures run to the cap.

Prompt	FPGA (IF4/bf16)	CUDA (bf16)	Δ steps	Agree
0	✓ 94	✓ 95	−1	✓
1	✓ 160	✓ 126	+34	✓
2	✓ 123	✓ 115	+8	✓
3	✓ 108	✓ 109	−1	✓
4	× 230	× 230	—	✓
5 [†]	× 290	× 290	—	✓
6	✓ 121	✓ 120	+1	✓
7	× 230	× 230	—	✓
8	✓ 107	✓ 106	+1	✓
9	× 230	× 230	—	✓
Rate	6/10	6/10		10/10

[†]Prompt 5 uses the `libero_object` variant selected for this evaluation set; both backends run the same task definition. Outcome agreement is 10/10. Five of the six successful episode pairs finish within eight simulation steps of one another; the remaining pair differs by 34 steps.

4 Throughput

Table 5 compares policy compute latency across platforms, with the ASIC projections in bold. Table 6 showcases the measured per-stage efficiency of the Unified Engine implementation. All platforms run batch 1 and ten denoise steps.

FPGA. Eight Unified Engine instances run on one 8 GB Alveo U50 at 366 MHz. Times come from the engine’s own cycle counters and are pure back-to-back execution; programs are compiled once up front and that one-time cost is excluded. The complete end-to-end run sustains a high 85% of peak compute throughput.

ASIC projection. The ASIC figures are derived from the measured FPGA cycle counts. $\pi_{0.5}$ is heavily compute-bound, so performance scales nearly linearly with engine count $\times$ clock across these configurations. The rows apply $(1600 \cdot N)/(366 \cdot 8)$: $26.2\times$ for $N=48$ engines and $35.0\times$ for $N=64$ at 1.6 GHz. Neither configuration reaches the 104 GB/s memory-bandwidth limit, so memory bandwidth does not constrain these projections.

GPUs. The RTX 5070 row is the measured fused `policy.infer` path in openpi JAX/XLA. The Jetson Orin Nano row is measured compute time from openpi PyTorch with staged bf16 execution; weight-staging time is excluded. Its deployed prefix is 968 tokens, compared with 832 rows on the U50. The Jetson Thor row is NVIDIA’s published result.

Table 5: Policy compute latency per inference, batch 1, ten denoise steps. ASIC rows (bold) are near-linear, compute-bound projections from the measured Alveo U50.

Platform	Config	Precision	Latency (ms)	inf/s	Price
AMD Alveo U50	8× Unified Engine, 366 MHz, 8 GB	IF4/bf16	13 100	0.076	FPGA dev kit
Unified Engine ASIC	48× Unified Engine, 1.6 GHz	IF4/bf16	500	2.00	<\$100
Unified Engine ASIC	64× Unified Engine, 1.6 GHz	IF4/bf16	375	2.67	<\$100
NVIDIA Jetson Thor T5000	MAXN, 130 W	bf16	132	7.58	\$5 499
NVIDIA Jetson Orin Nano 8 GB	7 W mode [‡]	bf16	4 266	0.234	\$399
NVIDIA RTX 5070	250 W	bf16	130	7.68	\$699

Latency values for U50, RTX 5070, and Jetson Orin Nano are measured; ASIC values are projections; Jetson Thor is NVIDIA-published. [‡]Orin excludes weight-staging time. Prices are reference comparison values; the ASIC figures are target device-cost estimates.

Table 6: Unified Engine efficiency on the Alveo U50 (mixed IF4/bf16), 8 GB, 8× Unified Engine at 366 MHz, 832-row prefix. Measured peak: 8×128 OPS/cycle at 366 MHz = 374.8 GFLOP/s. GFLOP/s counts hardware-issued work after padding; Table 1 reports the model’s true-shape work.

Section	Time (s)	GFLOP/s	% peak
Vision encoder	2.32	297.6	79
Prefix (Gemma-2B)	10.03	340.0	91
Action expert	0.75	101.9	27
End-to-end	13.1	318.0	85

Measured throughput: 0.076 inference/s.